

ENAE644 Term Project

Vai Srivastava¹

Abstract—This project implements and evaluates adversarial motion planning algorithms in a two-agent scenario where a deceptive agent attempts to reach a hidden goal while concealing its intent, and an interceptor agent seeks to infer the hidden goal and intercept the deceptive agent. The deceptive agent employs Adversarial RRT*, a sampling-based planner that extends RRT* by incorporating a learned deception cost function. A recurrent neural network serves as a surrogate observer, and the planner balances path optimality against observer classification accuracy using a weighted cost function. The interceptor agent combines inverse reinforcement learning to recover a behavioral model of the deceptive agent from historical demonstrations, particle filtering for online belief distribution tracking over candidate goals, and game-theoretic model predictive control for real-time interception planning with replanning. Both agents operate in a continuous two-dimensional workspace and are implemented in Python using JAX and Equinox for differentiable programming and GPU acceleration.

The full system was implemented end-to-end, encompassing neural network training, adversarial planning, and empirical evaluation. In the primary adversarial scenario, the deceptive agent successfully reached its goal in 2.30 s with a path length 12.4% above the optimal, while the interceptor was kept at a minimum distance of 2.50 units. However, the RNN observer achieved perfect classification accuracy (1.000) for all tested deception weights $\alpha \in \{0.0, 0.25, 0.5, 0.75, 1.0\}$, indicating that the three well-separated candidate goals in the experimental workspace were trivially distinguishable regardless of trajectory shaping. The principal finding is that geometric goal separability—not the deception algorithm’s parameterization—was the dominant factor limiting deceptive effectiveness, pointing to workspace and goal-set design as a critical but often overlooked dimension in adversarial motion planning research.

I. INTRODUCTION

The goal of this project is to implement and evaluate algorithms from two complementary sub-fields of adversarial motion planning. The first is *deceptive motion planning*, which is concerned with generating trajectories that conceal the agent’s true objective from an observer [1], [2]. The second is *goal identification and interception*, which seeks to infer an observed agent’s hidden objective and plan a trajectory to intercept it [3], [4]. Rather than studying these in isolation,

the project will pit a deceptive planner against an interception-focused planner in a shared environment, providing a direct adversarial evaluation of each algorithm’s strengths and limitations. Both agents will employ planning methods at least as sophisticated as RRT*, incorporating learning-based and game-theoretic components on top of their respective base planners.

II. MOTIVATION

Deceptive motion planning has direct applications in video game AI, where non-player characters must exhibit unpredictable yet purposeful behavior [3], [4]. However, the relevance of this problem extends well beyond entertainment into real-world domains such as strategic logistics, privacy-preserving navigation, and military operations. The ability to move deceptively can be the difference between mission success and failure [1], [2], while the complementary problem of identifying and intercepting deceptive agents is equally critical: security and defense systems must be capable of recognizing when an observed agent is behaving deceptively and responding accordingly [3], [5]. Studying the interplay between deception and counter-deception will provide insight into the robustness of each approach and reveal failure modes that would not surface when evaluating either in isolation.

It is important to note that prior work has explored this coupling in the context of autonomous vehicles [6] and multi-agent navigation [7], [8], but an adversarial pitting in a strategy-game context remains underexplored.

III. RELATED WORK

A. Deceptive Motion Planning

A central challenge in deceptive motion planning is balancing path cost against unpredictability. Nichols et al. [1] address this trade-off with Adversarial RRT*, a sampling-based planner that incorporates a learned deception cost into the RRT* objective. Their approach uses a recurrent neural network as a surrogate observer and penalizes trajectories that the network can classify correctly, achieving low observer accuracy while keeping path length near-optimal. Xu et al. [2] take a complementary approach grounded in probabilistic goal recognition, formalizing deceptive path planning as a mixed-integer program that maximizes goal ambiguity with respect to an observer’s belief distribution. Both methods assume a passive observer and evaluate deception in isolation rather than

¹Srivastava is with the Department of Aerospace Engineering, University of Maryland, College Park, MD 20742, The United States of America, vaisriv@umd.edu.

against an active adversary, which is the gap this project aims to address.

B. Goal Recognition and Interception

On the opposing side of the problem, several works focus on inferring an agent's hidden objective and acting on that inference. Xu and Yin [5] introduce *relative goal uncertainty*, an entropy-based metric that quantifies the goal-related information contained in each action, and show how it can be used to control the goal identification process from either side of the interaction. Zeng and Xu [3] extend goal recognition to explicitly account for deceptive agents by using inverse reinforcement learning to recover a behavioral model from the deceiver's historical data, then constructing a game-theoretic interference strategy. In the domain of first-person shooter games, Tastan et al. [4] learn player-specific motion models via inverse reinforcement learning and use particle-filter-based prediction to intercept opponents on partially occluded maps. These works provide the algorithmic foundations for the interceptor agent in this project, particularly the combination of learned behavioral models with online prediction.

C. Game-Theoretic and Interactive Planning

A related body of work frames multi-agent planning as a dynamic game in which each agent's strategy depends on the others'. Le Cleac'h et al. [8] propose LUCIDGames, which pairs an unscented Kalman filter for online estimation of other agents' cost functions with a receding-horizon game-theoretic planner, demonstrating real-time performance in autonomous driving scenarios. Espinoza et al. [6] tightly couple prediction and planning through a game-theoretic MPC that uses a multi-agent neural network policy as its predictive model, producing interactive behaviors between an autonomous vehicle and surrounding traffic. While both of these works operate in the autonomous driving domain, they demonstrate the value of jointly reasoning about one's own plan and other agents' likely responses, directly informing the adversarial evaluation framework in this project.

D. Adversary-Aware Motion Planning

Most closely related to the present work, Netter and Vamvoudakis [7] propose a motion planning framework in which a player agent navigates a multi-agent environment while simultaneously identifying and avoiding potential adversaries using Gaussian process classification. Their method includes real-time replanning to avoid likely adversarial agents and distinguishes adversaries from benign agents to prevent unnecessary evasive maneuvers. This project builds on a similar premise but inverts the emphasis: rather than treating interception avoidance as a byproduct of adversary classification, the deceptive agent here will actively minimize observer accuracy as a first-class planning objective, while the intercep-

tor will be a fully autonomous adversary rather than a fixed behavioral model.

IV. FORMAL PROBLEM DEFINITION

This section formalizes the adversarial motion planning problem as a two-agent scenario in which a deceptive agent attempts to reach a hidden goal while concealing its intent, and an interceptor agent seeks to infer the hidden goal and intercept the deceptive agent before the goal is reached.

A. Workspace and Agent Models

We consider a bounded, continuous two-dimensional workspace $\mathcal{W} \subset \mathbb{R}^2$ containing a static obstacle set $\mathcal{O} \subset \mathcal{W}$. The collision-free workspace is defined as $\mathcal{W}_{\text{free}} = \mathcal{W} \setminus \mathcal{O}$. Two agents operate within this workspace: a deceptive agent (Agent D) and an interceptor agent (Agent I).

1) *Deceptive Agent (Agent D)*: The deceptive agent's state at time t is its position $x_D(t) \in \mathcal{W}_{\text{free}}$. Agent D has a finite set of candidate goals $\mathcal{G}_D = \{g_1, g_2, \dots, g_M\} \subset \mathcal{W}_{\text{free}}$ known to both agents, and a true goal $g^* \in \mathcal{G}_D$ that is hidden from Agent I. The agent executes a trajectory $\xi_D : [0, T] \rightarrow \mathcal{W}_{\text{free}}$ satisfying the boundary conditions $\xi_D(0) = x_{D,0}$ and $\xi_D(T) = g^*$, where $x_{D,0}$ is the initial position and T is the time horizon. The trajectory is subject to kinodynamic constraints including velocity bounds and acceleration limits, which we denote generically as $\xi_D \in \Xi_D$, where Ξ_D is the set of kinodynamically feasible trajectories.

2) *Interceptor Agent (Agent I)*: The interceptor agent's state at time t is its position $x_I(t) \in \mathcal{W}_{\text{free}}$. Agent I starts at position $x_{I,0}$ and executes a trajectory $\xi_I : [0, T] \rightarrow \mathcal{W}_{\text{free}}$ with $\xi_I(0) = x_{I,0}$. The objective of Agent I is to infer the true goal g^* from observations of Agent D's partial trajectory and plan an interception trajectory to reach the predicted goal before Agent D. Agent I's trajectory is also subject to kinodynamic constraints $\xi_I \in \Xi_I$, which may differ from those of Agent D.

3) *Observer Model*: Agent I maintains a belief distribution over the candidate goals, represented as $b_t : \mathcal{G}_D \rightarrow [0, 1]$ where $\sum_{g \in \mathcal{G}_D} b_t(g) = 1$. This belief is updated incrementally based on observations of the partial trajectory $\xi_D[0, t]$. The belief update mechanism will be specified in detail in the following subsections.

B. Deceptive Motion Planning Problem

The deceptive agent seeks to generate a trajectory ξ_D^* that satisfies three competing objectives: (1) reach the true goal g^* , (2) minimize path cost, and (3) maximize the observer's uncertainty about g^* . This multi-objective optimization problem is formalized as

$$\xi_D^* = \arg \min_{\xi_D \in \Xi_D} [\alpha \cdot J_{\text{path}}(\xi_D) + (1 - \alpha) \cdot J_{\text{deception}}(\xi_D)] \quad (1)$$

where $J_{\text{path}}(\xi_D)$ is the path cost, $J_{\text{deception}}(\xi_D)$ quantifies the “revealingness” of the trajectory (i.e., how easily an observer can infer the true goal), and $\alpha \in [0, 1]$ is a weighting parameter that balances path optimality against deception.

The path cost is typically defined as the total path length or control effort:

$$J_{\text{path}}(\xi_D) = \int_0^T \|\dot{\xi}_D(t)\| dt \quad (2)$$

The deception cost is more subtle and can be formulated in multiple ways. Following the Adversarial RRT* framework [1], we employ a learned classifier f_θ (specifically, a recurrent neural network) as a surrogate observer. The classifier maps partial trajectories to probability distributions over goals: $f_\theta : \xi_D \mapsto \Delta(\mathcal{G}_D)$, where $\Delta(\mathcal{G}_D)$ denotes the probability simplex over $\mathcal{G}_D$. The deception cost can then be defined as the negative entropy of the classifier’s output:

$$J_{\text{deception}}(\xi_D) = -H(f_\theta(\xi_D)) \quad (3)$$

where $H(p) = -\sum_{g \in \mathcal{G}_D} p(g) \log p(g)$ is the Shannon entropy. High entropy corresponds to high uncertainty (good deception), so minimizing $-H$ is equivalent to maximizing uncertainty. Alternatively, the deception cost can be defined directly as the classifier’s accuracy:

$$J_{\text{deception}}(\xi_D) = P_\theta(\text{correct goal} \mid \xi_D) = f_\theta(\xi_D)(g^*) \quad (4)$$

An alternative formulation based on probabilistic goal recognition [2], [5] defines the deception cost as the negative goal ambiguity:

$$J_{\text{deception}}(\xi_D) = -\sum_{g \in \mathcal{G}_D} P(g \mid \xi_D) \log P(g \mid \xi_D) \quad (5)$$

where the posterior probability $P(g \mid \xi_D)$ is computed using a probabilistic goal recognition framework that models the likelihood of trajectories under different goal hypotheses.

The optimization problem is subject to the following constraints:

- Collision avoidance: $\xi_D(t) \in \mathcal{W}_{\text{free}}$ for all $t \in [0, T]$
- Kinodynamic feasibility: $\xi_D \in \Xi_D$ (velocity and acceleration limits)
- Boundary conditions: $\xi_D(0) = x_{D,0}$ and $\xi_D(T) = g^*$

1) *Solution Method*: We employ Adversarial RRT* [1], a sampling-based motion planner that extends the RRT* algorithm by incorporating the deception cost $J_{\text{deception}}$ into the cost function used for tree rewiring and path selection. The planner iteratively samples configurations, extends the search tree, and evaluates candidate paths using the combined cost $\alpha J_{\text{path}} + (1 - \alpha) J_{\text{deception}}$, producing trajectories that balance path optimality with unpredictability.

C. Goal Inference and Interception Problem

The interceptor agent must solve two interrelated subproblems:

- 1) Infer the deceptive agent’s true goal g^* from observations of the partial trajectory $\xi_D[0, t]$

- 2) Plan an interception trajectory to reach the predicted goal before Agent D.

1) *Goal Inference Subproblem*: The goal inference problem is addressed through a combination of offline learning and online belief updates.

a) *Inverse Reinforcement Learning (IRL) Phase*: Prior to runtime, Agent I uses inverse reinforcement learning to recover a behavioral model of Agent D from historical trajectory data. The IRL problem seeks to recover a reward (or cost) function $R_D(s, a)$ such that the observed trajectories are approximately optimal under this reward function. Following [3], we model the deceptive agent’s behavior by learning a reward function that captures both path efficiency and deception, enabling Agent I to predict likely future actions given a goal hypothesis.

b) *Online Belief Update*: During execution, Agent I maintains and updates the belief distribution $b_t(g)$ over candidate goals using Bayes’ rule:

$$b_{t+1}(g) \propto P(\xi_D[t, t+1] \mid g, \xi_D[0, t]) \cdot b_t(g) \quad (6)$$

where $P(\xi_D[t, t+1] \mid g, \xi_D[0, t])$ is the likelihood of the observed trajectory segment given goal hypothesis g and the trajectory history. This likelihood is computed using the behavioral model learned via IRL.

c) *Particle Filter Implementation*: Following [4], we employ a particle filter to maintain and propagate the belief distribution. Each particle represents a hypothesis about the true goal and the agent’s current state. The particle filter alternates between a prediction step (using the learned motion model to propagate particles forward) and an update step (reweighting particles based on observed trajectory segments).

The predicted goal at time t is obtained by selecting the maximum a posteriori (MAP) estimate:

$$\hat{g}(t) = \arg \max_{g \in \mathcal{G}_D} b_t(g) \quad (7)$$

2) *Interception Planning Subproblem*: Given the predicted goal $\hat{g}(t)$, Agent I must plan a trajectory to intercept Agent D. The interception planning problem is formulated as:

$$\xi_I^* = \arg \min_{\xi_I \in \Xi_I} T_{\text{intercept}} \quad (8)$$

subject to the constraint that Agent I reaches the predicted goal $\hat{g}$ before Agent D. However, because the prediction $\hat{g}(t)$ evolves over time as new observations are incorporated, a static plan is insufficient.

a) *Game-Theoretic Model Predictive Control (MPC)*: Following [8] and [6], we employ a receding-horizon game-theoretic planner. At each time step t , Agent I solves the finite-horizon optimization problem:

$$\min_{\xi_I[t, t+H]} J_I(\xi_I, \hat{\xi}_D) \quad (9)$$

where H is the prediction horizon and $\hat{\xi}_D$ is the predicted future trajectory of Agent D, computed by forward-simulating the learned behavioral model conditioned on the current belief distribution. The cost function J_I is defined as:

$$J_I(\xi_I, \hat{\xi}_D) = \|\xi_I(T_{\text{intercept}}) - \hat{g}\|^2 + \int_t^{t+H} \|\dot{\xi}_I(\tau)\|^2 d\tau \quad (10)$$

where the first term penalizes distance to the predicted goal and the second term penalizes control effort. The solution yields a planned trajectory for the next H time steps, of which only the first segment is executed before replanning at the next time step.

b) Replanning: The MPC framework enables real-time replanning. As Agent I observes new segments of Agent D's trajectory, the belief distribution $b_t(g)$ is updated, the predicted goal $\hat{g}(t)$ may change, and the interception plan is recomputed accordingly.

D. Adversarial Game Formulation

The interaction between the deceptive agent and the interceptor can be formalized as a two-player dynamic game with asymmetric information.

1) Players and Objectives:

- Player 1 (Agent D): Minimizes $J_D = \alpha J_{\text{path}} + (1 - \alpha) J_{\text{deception}}$
- Player 2 (Agent I): Minimizes J_I (interception cost, including time-to-intercept and control effort)

2) Information Structure: This is an asymmetric information game. Agent D has complete information: it knows the true goal g^* , the candidate goal set $\mathcal{G}_D$, and can observe Agent I's current state $x_I(t)$. In contrast, Agent I has incomplete information: it knows the candidate goal set $\mathcal{G}_D$ and can observe the partial trajectory $\xi_D[0, t]$, but does not know the true goal g^* . Agent I must infer g^* from observations.

3) Solution Concept: Due to the information asymmetry and the sequential nature of the interaction, this problem does not admit a standard Nash equilibrium solution. Instead, it is more appropriately modeled as a Stackelberg game (leader-follower game), in which Agent D (the leader) plans its trajectory anticipating that Agent I (the follower) will respond optimally given its observations. However, Agent D must account for the fact that Agent I does not know g^* and will update its beliefs and replan based on observed trajectory segments.

a) Adversarial Evaluation Framework: Rather than solving analytically for an equilibrium, we adopt an empirical adversarial evaluation framework. We implement Adversarial RRT* for Agent D and the IRL-based particle filter with game-theoretic MPC for Agent I, then evaluate their performance through simulation. Key performance metrics include:

- For Agent D: Observer classification accuracy at the time of goal completion, $P_\theta(g^* | \xi_D)$, and path length ratio $J_{\text{path}}(\xi_D^*)/J_{\text{path}}(\xi_D^{\text{opt}})$ where ξ_D^{opt} is the optimal (shortest) path
- For Agent I: Goal inference accuracy $\mathbb{1}[\hat{g}(T) = g^*]$ and time-to-intercept relative to Agent D's arrival time

- Overall: Success rate for each agent—does Agent D successfully reach g^* before being intercepted, or does Agent I successfully intercept Agent D?

b) Interaction Dynamics:

The game proceeds as follows. Agent D generates a deceptive trajectory using Adversarial RRT*. Agent I observes the trajectory incrementally and updates its belief distribution over goals using the IRL-learned behavioral model and particle filtering. Based on the updated belief, Agent I replans its interception trajectory using game-theoretic MPC. This cycle continues until one of two termination conditions is met: (a) Agent D reaches the true goal g^* , or (b) Agent I successfully intercepts Agent D (i.e., $\|x_I(t) - x_D(t)\| < \varepsilon_{\text{intercept}}$ for some small threshold $\varepsilon_{\text{intercept}}$).

V. METHODOLOGY

A. Overview

The project is implemented in Python, chosen for its extensive ecosystem of machine learning and scientific computing libraries. In particular, we use JAX (via the Equinox library) as the primary framework for differentiable programming, automatic differentiation, and GPU-accelerated computation.

The core of the project involves two competing agents in a continuous two-dimensional workspace. The first agent employs a deceptive motion planning algorithm building on Adversarial RRT* [1], which augments the sampling-based RRT* planner with a learned deception cost to generate trajectories that minimize an observer's ability to infer the agent's true goal. Entropy-based deceptive planning techniques [2], [5] inform the design of the deception objective. The second agent utilizes an identification and interception strategy, drawing on techniques from inverse reinforcement learning and game-theoretic prediction [3], [4], [8], with the objective of recognizing the deceptive agent's true goal and planning an intercept trajectory in real time. The approach of coupling adversary identification with reactive planning follows the framework proposed by Netter and Vamvoudakis [7], adapted to the adversarial evaluation setting. Both agents thus operate well beyond basic shortest-path planning: the deceptive agent layers a learned deception metric on top of RRT*, while the interceptor combines online goal inference with reactive replanning.

Of note, despite several of the referenced algorithms being originally developed for three-dimensional environments (e.g., first-person shooter game maps [4]), we work in a continuous two-dimensional workspace for ease of development and in the interest of completing the project within the semester. The simulation environment is custom-built in Python, providing full control over the experimental setup and enabling systematic evaluation of each algorithm's performance under varying conditions.

B. Implementation

The system was implemented end-to-end in Python using JAX and Equinox. A functional programming approach was adopted throughout: trajectories are represented as JAX pytrees, enabling automatic differentiation and JIT compilation, and neural network parameters follow Equinox conventions for clean separation of static structure and learnable leaves.

The shared infrastructure layer provides the geometric primitives on which both agents depend. The workspace representation supports circular and convex polygon obstacles with JIT-compiled collision detection using point-to-segment distance tests and ray-casting for polygon containment. Trajectory data structures store time-parameterized position and velocity arrays with linear interpolation for continuous-time queries, path length computation, and partial trajectory extraction. The agent controller interface defines a common action type used by both simplified test controllers and the full adversarial agents.

The deceptive agent’s planning system consists of three coupled components. The surrogate observer is a GRU-based recurrent neural network that maps a variable-length sequence of 2D positions to a probability distribution over candidate goals. It is trained offline on synthetic goal-directed trajectories generated by a standard (non-deceptive) variant of the planner, using cross-entropy loss with the Adam optimizer. The deception cost function consumes the observer’s output and computes either the negative entropy $-H(f_{\theta(\xi)})$ (entropy mode, where lower entropy means the observer is more confident) or the classifier probability assigned to the true goal $f_{\theta(\xi)}[g^*]$ (accuracy mode). The Adversarial RRT* planner wraps these components inside an asymptotically optimal sampling-based search: at each iteration it samples a configuration, steers toward it, evaluates the combined cost $\alpha J_{\text{path}} + (1 - \alpha) J_{\text{deception}}$ over candidate parent paths, adds the best node, and rewires nearby nodes. To avoid repeated JIT recompilation on growing path arrays, the observer input is truncated to a fixed window of the most recent 20 positions; rewiring decisions use path-length cost only to avoid calling the observer once per neighboring node.

The interceptor agent’s inference and planning system comprises three layers. A learned reward function—a three-layer MLP mapping (state, action) pairs to a scalar reward—is trained offline via a contrastive noise-estimation objective: for each expert (state, action) step the loss maximizes the reward relative to eight uniformly-sampled random actions. At runtime, a particle filter maintains a weighted ensemble of goal hypotheses; each particle’s weight is updated proportionally to the likelihood computed from the learned reward function, with systematic resampling when the effective sample size falls below half the particle count. A game-theoretic model predictive controller plans Agent I’s motion by computing a

belief-weighted expected trajectory for Agent D (by forward-simulating the learned reward policy for each candidate goal), then running Adam gradient descent on a horizon-length control sequence to minimize the tracking cost.

The simulation controller ties both agents together. Agent D executes its pre-planned trajectory by interpolating waypoints at each timestep; Agent I updates its particle filter belief, solves the MPC problem, and integrates the resulting velocity command. The episode terminates when Agent D reaches its true goal (within 0.5 units) or Agent I intercepts Agent D (agents within 0.5 units of each other).

A two-stage pipeline separates training from evaluation. The training stage generates synthetic datasets, trains both neural networks, and serializes the resulting model weights to disk using Equinox’s leaf serialization. The evaluation stage loads the saved checkpoints, runs the full adversarial simulation, records trajectories and metrics, and produces all figures.

C. Procedure

Experiments were conducted in a 10×10 workspace containing a single circular obstacle of radius 1.0 centered at (5, 5). Three candidate goals were placed at (9, 9), (9, 1), and (1, 9)—the three active corners of the workspace—so that each goal lies in a geometrically distinct region. The deceptive agent started at (1, 1) with true goal $g^* = (9, 9)$, and the interceptor started at (1, 9).

1) *Neural Network Training*: The observer network was trained on synthetic trajectories generated by a goal-directed RRT variant that plans without any deception objective. For each of the three candidate goals, 150 trajectories were generated from random starting positions sampled uniformly from the workspace, yielding a total training set of 450 trajectories. Training used cross-entropy loss with the Adam optimizer at a learning rate of 10^{-3} for 80 epochs with batch size 32.

The IRL reward network was trained on 200 expert demonstrations—goal-directed paths toward one of the three candidate goals—using a contrastive noise-estimation objective. For each consecutive (state, action) pair in a demonstration, the loss maximized the network’s reward on the expert action relative to eight uniformly-distributed random alternative actions sampled over $[0, 2\pi)$.

2) *Adversarial Simulation*: The primary adversarial trial was run with deception weight $\alpha = 0.5$. Agent D planned its full trajectory offline using Adversarial RRT* with 2000 iterations, step size 0.5, and goal bias probability 0.1. The interceptor updated its particle filter (200 particles) and solved the MPC optimization (horizon 15, 50 gradient steps) at each timestep $\Delta t = 0.1$ s. An episode ended when Agent D came within 0.5 units of its goal (Agent D win) or Agent I came within 0.5 units of Agent D (Agent I win), with a 60 s timeout.

3) *Deception Weight Ablation*: To isolate the effect of the deception parameter, a planning-only ablation sweep was

conducted over $\alpha \in \{0.0, 0.25, 0.5, 0.75, 1.0\}$. For each value, Agent D re-planned its trajectory using 300 RRT* iterations (reduced for speed), and the observer’s classification accuracy and path length ratio were recorded. The full simulation loop was not re-run per α ; instead, the same trained observer was applied to each planned trajectory to measure the deception effectiveness at planning time.

VI. RESULTS

A. Neural Network Training

The observer network converged rapidly. Cross-entropy loss dropped from 0.155 at epoch 0 to 7.6×10^{-5} by epoch 10, and reached numerical zero (to floating-point precision) by epoch 61, where it remained for all subsequent epochs. The IRL reward network exhibited slower, more gradual improvement over 500 training epochs. The contrastive loss decreased from 2.104 at epoch 0 to 1.822 after 40 epochs (a 13% reduction), then continued to decline—with a notable acceleration after epoch 250—ultimately reaching a final value of 0.933 at epoch 500, a total reduction of approximately 56%.

B. Primary Adversarial Trial

In the representative trial with $\alpha = 0.5$ —as displayed in Fig. 1—, Agent D successfully reached its true goal (9, 9) at $t = 2.30$ s. The deceptive trajectory spanned 12.7 units compared to an optimal straight-line distance of 11.3 units, yielding a path length ratio of 1.124.

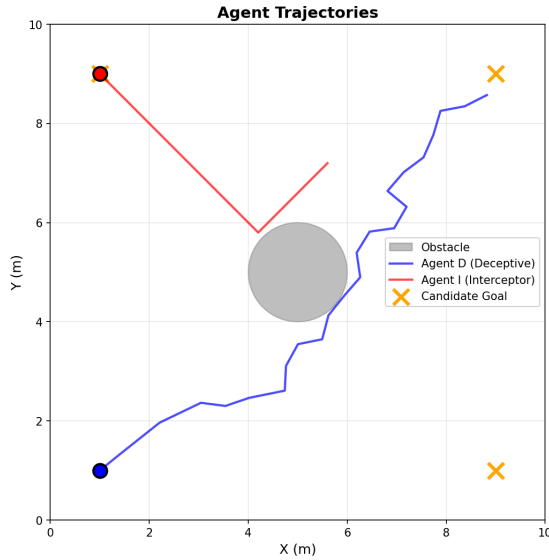


Fig. 1. Representative adversarial trial ($\alpha = 0.5$). Agent D (blue) navigates from (1, 1) to its true goal at (9, 9) using a deceptive path around the central obstacle. Agent I (orange) pursues Agent D but fails to intercept before the goal is reached. Stars mark the three candidate goals; the gray circle is the obstacle.

Agent I’s minimum approach distance to Agent D—charted with respect to time in Fig. 2—during the episode was 2.496 units, comfortably above the 0.5-unit interception threshold.

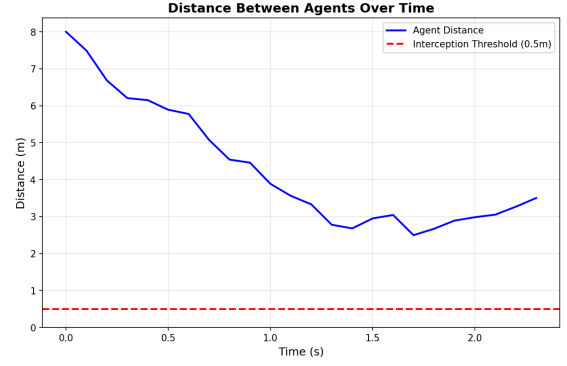


Fig. 2. Inter-agent distance as a function of time. The dashed horizontal line marks the 0.5-unit interception threshold. Agent I never approaches close enough to intercept Agent D, which reaches its goal at $t = 2.30$ s.

Despite Agent D’s win, the RNN observer assigned probability 1.000 to the true goal throughout the trajectory, indicating that the deceptive trajectory provided no measurable obfuscation to the surrogate observer. The particle filter’s MAP estimate equaled the true goal for 33% of timesteps (goal inference accuracy), reflecting transient ambiguity in the first few steps before the belief converged; time-to-convergence (first time the maximum belief exceeded 0.8) was 1.0 s. Both belief signals are shown as a function of simulation time in Fig. 3.

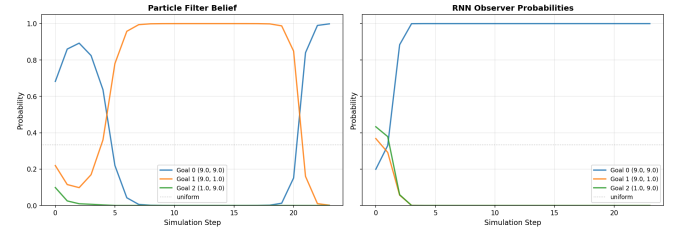


Fig. 3. Goal belief evolution over the episode. *Left*: particle filter belief distribution; the probability mass assigned to the true goal $g_0 = (9, 9)$ begins near 0.33 (uniform prior) and converges toward 1.0 after an initial period of ambiguity. *Right*: RNN observer probability distribution; the observer assigns probability 1.000 to the true goal from the very first observed positions, confirming perfect classification throughout the episode.

Performance metrics for the primary trial are summarized in Table I.

TABLE I. SUMMARY METRICS FOR THE PRIMARY ADVERSARIAL TRIAL ($\alpha = 0.5$).

Metric	Value
Winner	Agent D
Completion time (s)	2.30
Observer accuracy	1.000
Path length ratio	1.124
Minimum interception distance (units)	2.496
Goal inference accuracy	0.333
Time to belief convergence (s)	1.00
Deception effectiveness	0.445
Interception efficiency	0.224
Distance traveled—Agent D (units)	12.71
Distance traveled—Agent I (units)	6.50

C. Deception Weight Ablation

Fig. 4 shows the results of the planning-only ablation sweep across five values of α . Observer accuracy remained 1.000 for every tested value, confirming that goal classification accuracy is independent of the deception weight in this configuration. Path length ratios ranged from 1.061 ($\alpha = 0.75$) to 1.203 ($\alpha = 0.25$), with no consistent monotone relationship between α and path length. Deception effectiveness, computed as $\alpha(1 - \text{acc}) + (1 - \alpha)\left(\frac{1}{\text{plr}}\right)$, decreased from 0.875 at $\alpha = 0$ to 0.0 at $\alpha = 1$.

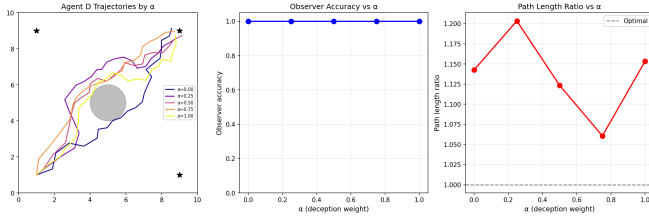


Fig. 4. Ablation results across deception weights $\alpha \in \{0.0, 0.25, 0.5, 0.75, 1.0\}$. *Left*: planned trajectories for each α (colors from purple to yellow). *Center*: RNN observer accuracy (constant at 1.000). *Right*: path length ratio showing the efficiency cost of deception.

Table II tabulates the per- α measurements. The constant observer accuracy means that the composite deception effectiveness score is driven entirely by the path length ratio term.

TABLE II. DECEPTION WEIGHT ABLATION RESULTS. OBSERVER ACCURACY IS 1.000 FOR ALL α VALUES.

α	Obs. Acc.	Path Len. Ratio	Dec. Eff.	Waypoints
0.00	1.000	1.143	0.875	23
0.25	1.000	1.203	0.623	25
0.50	1.000	1.123	0.445	23
0.75	1.000	1.061	0.236	21
1.00	1.000	1.154	0.000	24

VII. DISCUSSION

The most striking result is that the RNN observer achieved perfect classification accuracy across all tested conditions. The three candidate goals in the experiment are placed at well-separated corners of a 10×10 workspace—(9, 9), (9, 1), and (1, 9)—and any trajectory originating from (1, 1) that reaches one of these corners will exhibit strong directional commitment early in its execution. Even with $\alpha = 0$, where the Adversarial RRT* planner devotes the entire cost budget to deception, the RNN learned features sufficient for perfect discrimination. This outcome contrasts with the results reported by Nichols et al. [1], who observed observer accuracy dropping to roughly 10% on adversarially planned paths (compared to 46% on optimal paths) when goals were arranged with greater geometric ambiguity. The critical difference appears to be goal-set geometry: in their configuration, all candidate goals lie on the same arc from the start, so optimal paths toward different goals share a long common prefix, making deceptive rerouting effective. In our configuration, the goals are in three distinct quadrants, so even a heavily perturbed trajectory quickly reveals which quadrant it is heading toward.

This finding reveals a fundamental prerequisite for the deception algorithm: there must exist low-cost trajectory prefixes that are genuinely ambiguous with respect to multiple candidate goals. When goal separability is high, the RNN saturates at perfect accuracy regardless of the planner’s effort, and the deception-weight parameter α has no effect on observer confusion. The path length ratios in the ablation sweep (ranging from 1.061 to 1.203 with no consistent trend) further confirm that at $\alpha = 0$ the planner is spending additional planning cost on detours that do not fool the observer.

The interceptor’s performance reflects a different failure mode. Agent I successfully converged to the correct goal hypothesis within 1.0 s (at which point the episode was already more than one-third complete), but it was still unable to close to within interception distance. The minimum inter-agent distance of 2.496 units and an interception efficiency of 0.224 indicate that the MPC controller was unable to generate a control sequence aggressive enough to reach Agent D before goal completion. This is attributable to the MPC’s short planning horizon (15 steps) and the relatively short episode duration (2.30 s): by the time the belief had converged and the MPC had produced a meaningful intercept plan, Agent D was too close to its goal to be caught. Increasing the MPC horizon or giving the interceptor a higher maximum speed would likely invert the outcome.

Although the IRL reward network was trained for 500 epochs and achieved a total loss reduction of approximately 56% (from 2.104 to 0.933), the 33% goal inference accuracy and slow belief convergence indicate that the learned reward function did not produce strongly discriminative likelihoods.

In a continuous 2D workspace the negative samples (random unit-vector actions) are broadly distributed, making it relatively easy for the network to assign higher rewards to expert actions than to random ones—without necessarily capturing fine-grained directional preferences that would sharpen per-step likelihoods. The delayed convergence pattern observed during training (slow decrease through epoch 250, followed by acceleration) suggests the network eventually found a more discriminative regime, but the improvement was not sufficient to produce sharp particle filter beliefs within the short episode window. A more expressive IRL formulation—such as maximum entropy IRL with a discretized workspace or a flow-based policy model—would likely improve particle filter sharpness and goal inference accuracy.

VIII. CONCLUSION

This project implemented a complete adversarial motion planning system: a deceptive agent using Adversarial RRT* with a GRU-based surrogate observer, and an interceptor agent using contrastive IRL, particle filtering, and game-theoretic MPC. Both agents were trained end-to-end and evaluated in a shared continuous 2D workspace, with all training, planning, simulation, and visualization code implemented in JAX and Equinox.

The primary experimental finding is that Agent D won the adversarial episode, reaching its goal in 2.30 s while maintaining a safe distance of 2.50 units from Agent I. However, the deception mechanism failed on its own terms: the RNN observer classified Agent D’s true goal with probability 1.000 regardless of the deception weight α . This occurred because the experimental workspace’s three widely-separated candidate goals are geometrically distinguishable from any trajectory prefix, rendering the deception objective ineffective. The ablation sweep over α confirmed this: observer accuracy was flat at 1.000 for all five tested values.

The key limitation of this work is scale: a single trial, three candidate goals, and a simple workspace are insufficient to establish statistical conclusions about algorithm performance. Future work should evaluate on workspace configurations with high goal ambiguity (as used in [1]), extend to 3D environments, and conduct repeated trials with randomized initial conditions. On the interceptor side, replacing the contrastive IRL objective with full maximum entropy IRL and extending the MPC horizon would likely produce more competitive interception performance. More broadly, the framework would benefit from a closed-loop treatment in which Agent D replans dynamically in response to observed interceptor behavior, rather than executing a fixed offline trajectory.

REFERENCES

- [1] H. Nichols, M. Jimenez, Z. Goddard, M. Sparapany, B. Boots, and A. Mazumdar, “Adversarial Sampling-Based Motion Planning,” *IEEE Robotics and Automation Letters*, vol. 7, no. 2, pp. 4267–4274, 2022, doi: 10.1109/LRA.2022.3148464.
- [2] K. Xu, Y. Zeng, L. Qin, and Q. Yin, “Single Real Goal, Magnitude-Based Deceptive Path-Planning,” *Entropy*, vol. 22, no. 1, p. 88, 2020, doi: 10.3390/e22010088.
- [3] Y. Zeng and K. Xu, “Recognition and interfere deceptive behavior based on inverse reinforcement learning and game theory,” *Journal of Systems Engineering and Electronics*, vol. 34, no. 2, pp. 270–288, 2023, doi: 10.23919/JSEE.2023.000012.
- [4] B. Tastan, Y. Chang, and G. Sukthankar, “Learning to intercept opponents in first person shooter games,” in *2012 IEEE Conference on Computational Intelligence and Games (CIG)*, 2012, pp. 100–107. doi: 10.1109/CIG.2012.6374144.
- [5] K. Xu and Q. Yin, “Goal Identification Control Using an Information Entropy-Based Goal Uncertainty Metric,” *Entropy*, vol. 21, no. 3, p. 299, 2019, doi: 10.3390/e21030299.
- [6] J. L. V. Espinoza, A. Liniger, W. Schwarting, D. Rus, and L. V. Gool, “Deep Interactive Motion Prediction and Planning: Playing Games with Motion Prediction Models,” in *Proceedings of The 4th Annual Learning for Dynamics and Control Conference*, R. Firoozi, N. Mehr, E. Yel, R. Antonova, J. Bohg, M. Schwager, and M. Kochenderfer, Eds., in *Proceedings of Machine Learning Research*, vol. 168. PMLR, 2022, pp. 1006–1019. doi: 10.48550/arXiv.2204.02392.
- [7] J. Netter and K. G. Vamvoudakis, “Motion Planning with Adversary Avoidance Using Gaussian Process Classification,” in *2024 IEEE 63rd Conference on Decision and Control (CDC)*, 2024, pp. 2299–2304. doi: 10.1109/CDC56724.2024.10886561.
- [8] S. Le Cleac’h, M. Schwager, and Z. Manchester, “LUCIDGames: Online Unscented Inverse Dynamic Games for Adaptive Trajectory Prediction and Planning,” *IEEE Robotics and Automation Letters*, vol. 6, no. 3, pp. 5485–5492, 2021, doi: 10.1109/LRA.2021.3074880.